

Design Note: Low-Level Design (LLD) Practice Platform

Project: Takshaka — Low-Level Design Practice Platform

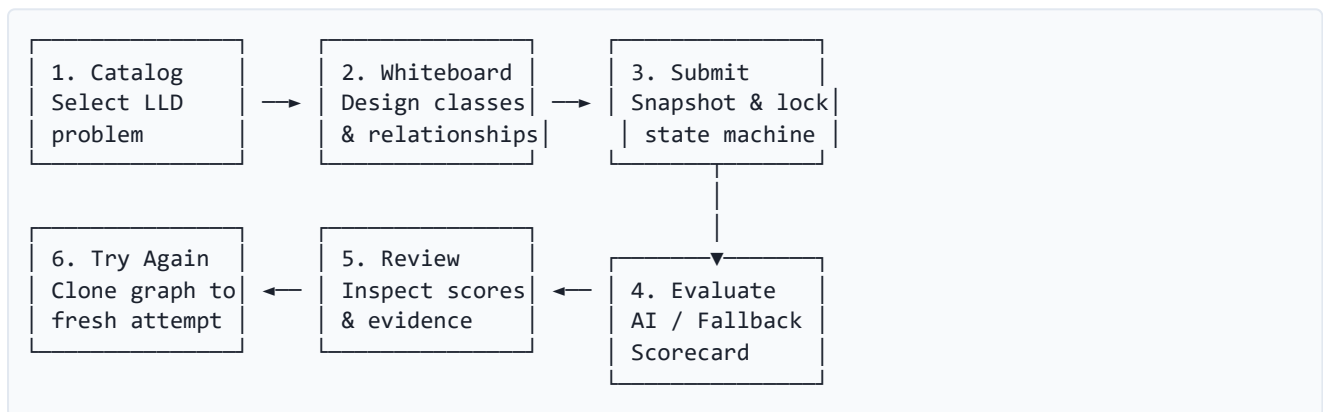
Deliverable: Design Note (2-Day Engineering Assignment, Section 7)

1. MVP Overview

Takshaka is an interactive, zero-setup platform designed to replicate real-world Low-Level Design interview practice. Candidates design domain architectures on an interactive UML canvas, submit them for automated evaluation against an authoritative rubric, and inspect structured scorecards to iterate their designs.

2. End-to-End User Flow

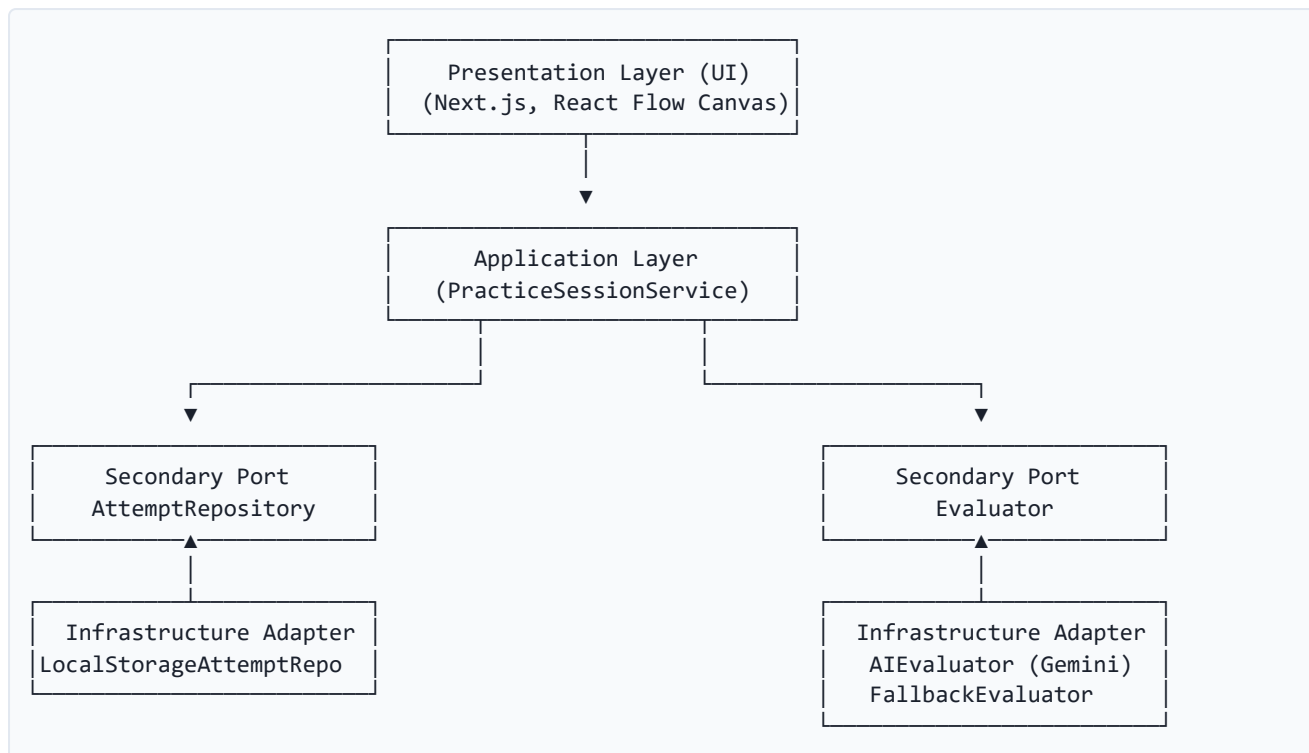
The platform enforces an intentional 6-step practice loop:



- Problem Discovery:** Candidate browses the catalog on the homepage and selects a problem (e.g. *Parking Lot* or *Elevator System*).
- Dual-Pane Workspace:** Left pane shows functional requirements and rubric criteria; right pane opens the interactive React Flow whiteboard.
- Whiteboard Modeling:** Candidate creates UML nodes (`class`, `interface`, `abstract class`, `enum`), adds attributes and methods, and connects relationships (`Inheritance`, `Implementation`, `Aggregation`, `Composition`, `Association`, `Dependency`).
- Submission:** Candidate clicks "Submit Architecture". The attempt transitions from `DRAFT` to `EVALUATING`, locking the canvas to prevent race conditions.
- Scorecard & Feedback:** Server evaluates the submission against the rubric and returns a structured `Feedback` scorecard (0–100 score, per-criterion scores with evidence, concerns, suggestions, strengths, and improvements).
- Iteration / History:** Completed attempts are saved immutably. Clicking "Try Again (Clone & Iterate)" spawns a new `DRAFT` attempt with the current graph, allowing candidates to address critique and re-submit.

3. Core Domain Model & Hexagonal Architecture

The platform strictly adheres to **Clean Architecture / Ports & Adapters**. The domain layer (`src/domain/`) has **zero dependencies** on UI frameworks, React, Next.js, or AI vendor SDKs.



Core Entities & Value Objects:

- **Problem (Entity)**: Owns problem requirements, constraints, and authoritative `RubricCriterion[]`.
- **Attempt (Aggregate Root)**: Manages lifecycle states (`DRAFT` → `EVALUATING` → `COMPLETED` / `FAILED`). Enforces invariants (e.g. completed attempts cannot be modified or re-completed).
- **Submission (Value Object)**: Contains immutable submission content (`format` , `content` , `submittedAt`). Decoupled from React Flow node structures via an Anti-Corruption Layer (`mapper.ts`).
- **Feedback (Value Object)**: Structured evaluation outcome (`overallScore` , `rubricEvaluations` , `strengths` , `improvements`).
- **RubricEvaluation (Value Object)**: Granular evaluation per criterion (`criterion` , `score` 1-5, `evidence` , `concern` , `suggestion` , `confidence`).

Outbound Ports (Interfaces):

- **Evaluator**: `evaluate(problem: Problem, submission: Submission): Promise`
- **AttemptRepository**: `save(attempt: Attempt): Promise` , `getById(id: string): Promise` , `listByProblemId(problemId: string): Promise`

4. Evaluation Approach & Strategy

Separation of Deterministic vs. Judgment Checks:

- **Deterministic**: Input schema validation (Zod), minimum entity count requirements, state machine lifecycle transitions, duplicate-submission guards, and fallback rubric scoring.
- **LLM-Powered Reasoning**: Architectural synthesis, evaluation of SOLID principles (SRP, OCP, LSP, ISP, DIP), separation of concerns, pattern identification (Strategy, Factory, State), and contextual feedback.

Zero-Hallucination Prompt Engineering:

- The evaluation engine sends the **authoritative problem rubric** directly in the system prompt.
- The model is strictly instructed:
 1. Evaluate *only* architectural design and relationships (not syntax).

- 2. Grade *every* rubric criterion.
- 3. Cite *concrete evidence* directly from the candidate's classes.
- 4. Return strictly valid JSON conforming to the `Feedback` schema.

Resilience & Offline Fallback:

- In `src/app/api/evaluate/route.ts`, if `GEMINI_API_KEY` or `OPENAI_API_KEY` is not present, or if external network calls fail, the system automatically delegates to `DeterministicFallbackEvaluator`, guaranteeing uninterrupted practice without runtime crashes.

5. Architectural Change Tests

The architecture was explicitly challenged with the two assignment change tests:

Change Test A: Evolving Submission Formats

- *Question:* Today the learner submits UML diagrams. Later, how would you support code or plain text?
- *Solution:* The domain `Submission` model is an abstract payload with a `format: SubmissionFormat` discriminator. To support code, we simply add a code editor adapter (e.g. Monaco) that emits `SubmissionFormat.CODE`. The `Attempt` aggregate, state machine, repository, and session services require **zero changes**.

Change Test B: Swapping the Evaluator Implementation

- *Question:* Today feedback comes from an AI evaluator. Later, how do you add a rule-based evaluator or human review without rewriting practice flow?
- *Solution:* The `Evaluator` port is an interface. `PracticeSessionService` and `/api/evaluate` depend solely on the interface. A human review evaluator (`HumanReviewEvaluator implements Evaluator`) or rule-based evaluator (`StaticAnalysisEvaluator implements Evaluator`) can be plugged in via dependency injection with **zero modification to the domain or UI flow**.

6. Key Engineering Trade-Offs

Decision	Alternative Considered	Rationale & Trade-Off
Visual Whiteboard Canvas	Monaco code editor (Java/C++)	Whiteboards test <i>spatial architectural thinking</i> and relationships directly, eliminating syntax/compilation debugging time in an interview setting.
Client-Side Persistence (localStorage)	PostgreSQL / Supabase	Provides zero-latency autosave and zero setup friction for candidates. Simplifies deployment to a serverless monolith without external database dependencies.
Strict JSON Rubric Prompting	Conversational Chatbot	Conversational chatbots hallucinate scores and give vague advice. Structured JSON guarantees measurable 0–100 scorecards, structured evidence, and repeatable retry benchmarking.
Synchronous Serverless Route	Distributed Task Queue (Redis/BullMQ)	In an MVP with single-digit concurrent users, an async queue adds massive operational complexity (Redis, worker fleets) for a 2–3 second AI call. A lightweight Next.js route keeps deployment practical and robust.